QUANSER
INNOVATE·EDUCATE

Concept Review

Microphone & Speaker

Why Use Speaker and Microphone?

Microphones capture vibrations in air and turn them into electrical and then digital signals, while a speaker performs the reverse. Together, they form the foundation of many real-world systems that listen, process, and respond, enabling applications such as speech recognition, voice communication, environmental monitoring, and acoustic sensing.

Sound Composition and Sine Waves

Sound is a vibration that propagates as a pressure wave through a medium (such as air). Microphones convert these pressure variations into electrical voltage signals, and speakers perform the reverse, converting electrical signals back into acoustic pressure waves.

The simplest form of sound is a pure tone, represented mathematically as a sine wave:

$$x(t) = A\sin(2\pi f t + \phi)$$

Where A is the amplitude (maximum pressure or voltage), f is frequency in Hz (cycles per second), t is time in seconds, and ϕ is phase offset in radians

A single sine wave is a pure tone as it contains only one frequency component. However, most real-world sounds (speech, music, noise) are composite signals consisting of many sine waves of different frequencies and amplitudes.

This is expressed by Fourier's theorem, which states that any periodic signal can be represented as a sum of sine and cosine waves:

$$x(t) = \sum_{k=1}^{N} A_k \sin(2\pi f_k t + \phi_k)$$

Here, each $f_k$corresponds to a frequency component in the sound. The fundamental frequency determines the pitch. Harmonics (integer multiples of the fundamental) shape the timbre or tone color. Noise or complex sounds contain a continuous range of frequencies.

Sampling and Digital Audio Fundamentals

When sound is recorded by a microphone, it is converted from a continuous (analog) voltage to discrete digital samples. This process involves sampling and quantization.

Sampling captures the signal amplitude at regular time intervals defined by the sampling rate (f_s):

$$x[n] = x(nT_s)$$

Where $x[n]$ is the discrete form of the signal $x(t)$, and T_s is the sampling period ($\frac{1}{f_s}$).

According to the Nyquist–Shannon sampling theorem, the sampling rate must be at least twice the highest frequency component in the signal to avoid aliasing. If this condition is not met, aliasing occurs — higher frequency components fold back into lower frequencies, distorting the signal.

Typical sample rate for telephone is 8 kHz, while CD-quality audio has a sample rate of 44.1 kHz. Nowadays, common professional and embedded applications are 48 kHz.

Each sampled value is approximated by the nearest available digital level. The number of possible levels depends on bit depth (N_b), number of levels = 2^{N_b}. A higher bit depth improves amplitude resolution and reduces quantization noise. For example, the classic 8-bit

sounds have coarse resolution (256 levels), while current standard audio streaming services use 16-bit sounds (65,536 levels).

In practice, audio data is processed in blocks (or frames) of samples, rather than one sample at a time. For a block size of N samples and a sample rate f_s, the duration of each block is:

$$T_{block} = \frac{N}{f_s}$$

Smaller block sizes lead to faster updates and lower latency, but higher CPU load, while larger block sizes create smoother visualization and better frequency resolution in FFT, but slower updates. These trade-offs are important for real-time applications like sound visualization and event detection.

Fast Fourier Transform (FFT)

While the time-domain representation of a signal $x(t)$or $x[n]$shows how amplitude varies over time, it often hides the *frequency content* that defines the sound's pitch, tone, and harmonic structure. Many important sound features are better understood by transforming the signal into the frequency domain, which shows how much of each frequency is present, as shown in Figure 1.

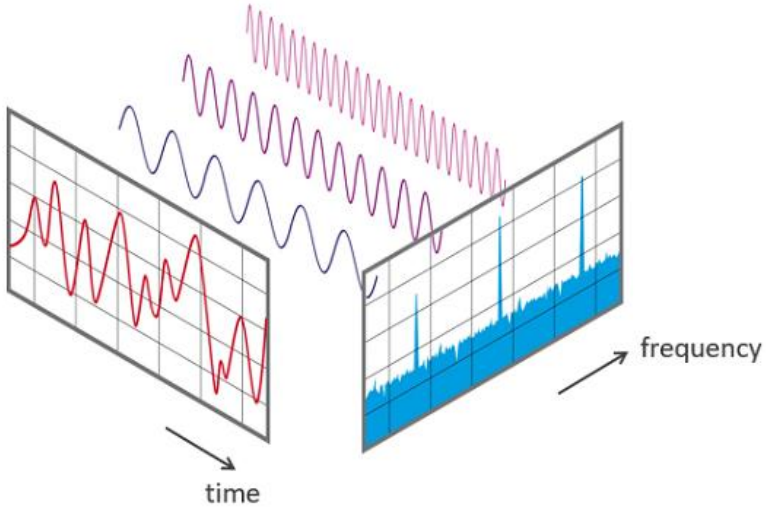

Figure 1. A complex signal composed of sinusoids in time and frequency domain.

For a continuous-time signal $x(t)$, Fourier Transform decomposes it into its frequency components. The transform projects the signal onto sinusoidal basis functions to determine how much of each frequency is present. In digital systems, we cannot handle continuous signals, so we use the **Discrete Fourier Transform (DFT)** to analyze sampled data $x[n]$, $n = 0,1,\ldots,N-1$:

$$X[k] = \sum_{n=0}^{N-1} x[n]\, e^{-\frac{i2\pi kn}{N}}, \qquad k = 0,1,\ldots,N-1$$

Where $x[n]$ is the time-domain signal, $X[k]$ is the DFT coefficient at frequency bin $f_k = k\Delta f$, with Δf being the frequency resolution $\Delta f = f_s/N$.

For example, at a sampling rate of $f_s = 48{,}000$ Hz and FFT block size $N = 1024$:

$$\Delta f = \frac{44100}{1024} \approx 43.1\text{Hz}$$

Since $X[k]$ is complex, it contains both magnitude and phase information:

$$|X[k]| = \sqrt{(\text{Re}\{X[k]\})^2 + (\text{Im}\{X[k]\})^2}$$
$$\angle X[k] = \tan^{-1}\left(\frac{\text{Im}\{X[k]\}}{\text{Re}\{X[k]\}}\right)$$

In practice, Magnitude spectrum shows how strong each frequency component is. Phase spectrum shows timing and alignment information, but it's often omitted for basic audio analysis.

The Fast Fourier Transform (FFT) is an efficient algorithm that computes the DFT. In practice, the FFT produces the same result as the DFT, but far more efficiently. In modern software environments, you rarely implement FFT manually, optimized libraries are used instead. In Python, the most common options are **numpy.fft**, **scipy.fft**, and **torch.fft**.

Solutions for teaching and research. Made in Canada.